EUSKO JAURLARITZA
GOBIERNO VASCO

ELORRIETA ERREKA MARI CIFP LHII

Desarrollo de aplicaciones en Python

Tema 2.6:
Hito 1 · Práctica integradora

Estructuras de control en Python

CC BY NC SA

Contenidos del tema

- Cierre de la UT2
- Práctica P2 · Sala de juegos
- Cómo trabajar
- Corrección comentada
- Lo que viene: funciones

Cierre de la UT2

Lo que hemos aprendido en la UT2

NECESITO…	HERRAMIENTA
Repetir mientras se cumpla una condición	**`while`**: contador, acumulador, centinela, bandera
Repetir N veces o recorrer una colección	**`for`** y **`range()`**
Salir antes, saltar una vuelta, saber si hubo salida	**`break`**, **`continue`** y **`else`** del bucle
Recorrer tablas y matrices	Bucles anidados
Índice, parejas, orden, claves y valores	**`enumerate`**, **`zip`**, **`sorted`**, **`items()`**
Crear listas y diccionarios en una línea	Comprensiones (con filtro y con ternario)
Encontrar y corregir errores	Traceback, depurador y tabla de traza
Buscar, máximo y mínimo, contar, validar	Algoritmos clásicos con bucles

La práctica de hoy usa **todas** estas herramientas en un único programa.

¿while o for?

while · no sé cuántas vueltas

Se repite **mientras** se cumpla una condición.

Menú que se repite hasta elegir salir.

Pedir un dato hasta que sea válido.

Jugar rondas hasta que alguien llegue a N victorias.

for · sé cuántas o recorro algo

Se repite **una vez por elemento** de una secuencia.

Como mucho 7 intentos: **`for intento in range(1, 8)`**.

Recorrer el historial de partidas.

Su **`else`** dice si el bucle terminó **sin `break`**.

En la P2 aparecen los dos, y uno **dentro** del otro: un **`for`** para los intentos y, dentro, un **`while`** que pide cada número hasta que es válido.

Autoevaluación: ¿qué muestra?

PREGUNTA 1

```
for i in range(3):
    if i == 5:
        break
else:
    print("fin")
```

PREGUNTA 2

```
n = 0
while n < 10:
    n += 3
print(n)
```

PREGUNTA 3

```
pares = [x * 2 for x in range(5)
         if x % 2 == 0]
print(pares)
```

PREGUNTA 4

```
notas = {"Ana": 7, "Luis": 4}
r = {n: "A" if v >= 5 else "S"
     for n, v in notas.items()}
print(r)
```

Práctica P2 · Sala de juegos

El programa: un menú que se repite

EJECUCIÓN

```
=== SALA DE JUEGOS ===
1. Adivina el número
2. Piedra, papel o tijera
3. Estadísticas
0. Salir
Opción: 1
Nivel (f, n, d): f
He pensado un número
del 1 al 50. Tienes 8 intentos.
Intento 1/8: 25
El número secreto es menor que 25.
```

1 Adivina el número

Tres niveles, pistas mayor o menor, intentos limitados.

2 Piedra, papel o tijera

Contra el ordenador, al mejor de 3 o de 5.

3 Estadísticas

Resumen de todas las partidas, hecho con comprensiones.

Sin funciones todavía: todo en una celda y sin `def`. Con las funciones la refactorizaremos.

Adivina el número

NIVEL	SE ESCRIBE	NÚMERO	INTENTOS
Fácil	**`f`**, **`fácil`**, **`facil`**	1 a 50	8
Normal	**`n`**, **`normal`**	1 a 100	7
Difícil	**`d`**, **`difícil`**, **`dificil`**	1 a 500	9

Reglas

`for` con **`range`** para los intentos y su **`else`** si se acaban.

Un dato no válido o **repetido** avisa y **no gasta intento**.

La pista se elige con una expresión condicional.

EJECUCIÓN

```
Intento 1/8: 25
El número secreto es menor que 25.
Intento 2/8: doce
Escribe un número entero del 1 al 50.
Intento 2/8: 12
El número secreto es menor que 12.
Intento 3/8: 12
Ya habías dicho ese número.
Intento 3/8: 6
El número secreto es mayor que 6.
Intento 4/8: 9
El número secreto es menor que 9.
Intento 5/8: 7
¡Acertaste en 5 intentos!
```

Piedra, papel o tijera

PUNTO DE PARTIDA

```
JUGADAS = ["piedra", "papel", "tijera"]
# clave gana a valor
GANA_A = {"piedra": "tijera", "papel": "piedra",
          "tijera": "papel"}
```

Reglas

Al mejor de 3 (gana quien llega a 2) o de 5 (a 3).

`random.choice(JUGADAS)` elige la jugada del ordenador.

`GANA_A[jugada] == ordenador` → ganas la ronda.

Los **empates no suman**: la partida puede alargarse.

EJECUCIÓN

```
¿Al mejor de cuántas rondas? (3 o 5): 3
Ronda 1 · piedra, papel o tijera: Piedra
Ordenador: tijera → ganas la ronda.
Marcador: 1-0
Ronda 2 · …: papel
Ordenador: papel → empate.
Marcador: 1-0
Ronda 3 · …: lagarto
Jugada no válida.
Ronda 3 · …: papel
Ordenador: piedra → ganas la ronda.
Marcador: 2-0
¡Has ganado la partida!
```

Estadísticas: el historial de partidas

ASÍ SE GUARDAN LAS PARTIDAS

```
# una tupla por partida:
# (juego, ganada, intentos_o_rondas)
historial = [("adivina", True, 7),
             ("ppt", True, 3),
             ("adivina", True, 8),
             ("ppt", False, 2),
             ("adivina", True, 5)]
```

Obligatorio

Tres comprensiones: con **filtro**, con **ternario** y de **diccionario**.

Mejor partida y favorita con el algoritmo de máximo y mínimo.

SALIDA

```
=== ESTADÍSTICAS ===
Partidas: 5 · ganadas: 4 (80 %)
Resultados: G G G P G
Adivina el número · partidas: 3
  · ganadas: 3
Piedra, papel o tijera · partidas: 2
  · ganadas: 1
Mejor partida de Adivina:
  la nº 5 (5 intentos)
Tus jugadas: piedra 1, papel 2,
  tijera 2
Tu jugada favorita: papel
```

Qué se entrega y cómo se valora

CRITERIO	PUNTOS
Diagramas de flujo con los bucles bien dibujados	2
Menú en bucle con `match-case`	1
Validación de todas las entradas	1,5
Adivina el número (con `for-else`)	2
Piedra, papel o tijera	1,5
Estadísticas con comprensiones	1
Pruebas, Parte D y estilo	1

En el notebook

A · Dos diagramas, **antes** de programar.

B · El código en una celda.

C · 12 casos de prueba como mínimo.

D · ¿Qué código se repite?

Práctica **formativa**: no cuenta para la nota final, pero se corrige y se comenta. Entrega del enlace en Moodle.

Cómo trabajar

De la idea al programa en cinco pasos

1

Diagramas

Menú y Adivina el número, en draw.io. Revisión antes de seguir.

2

Esqueleto

Menú en bucle con las opciones vacías. Comprueba que sales.

3

Adivina

Nivel, intentos, validación y for-else.

4

Piedra, papel o tijera

Mejor de N, empates y marcador.

5

Estadísticas y pruebas

Comprensiones, máximos y tabla de pruebas.

Prueba **cada paso** antes de empezar el siguiente. Con **`MODO_PRUEBA = True`** sabrás el número secreto y la jugada del ordenador.

Empieza por el esqueleto

```
while True:
    print("\n=== SALA DE JUEGOS ===")
    # ... resto del menú ...
    opcion = input("Opción: ").strip().lower()
    match opcion:
        case "1":
            print("Adivina: en construcción")
        case "2":
            print("PPT: en construcción")
        case "3":
            print("Estadísticas: en construcción")
        case "0" | "s" | "salir":
            print("¡Hasta pronto!")
            break
        case _:
            print("Opción no válida.")
```

Antes de seguir

Prueba las opciones 1, 2, 3, x, « 2 », S y salir.

`historial` y **`jugadas_usuario`** se crean **antes** del **`while`**: si no, se borran en cada vuelta.

Después, sustituye cada **`print`** por el juego, uno a uno.

Corrección comentada

Errores típicos

1 Convertir antes de validar

`int(texto)` con `doce` rompe el programa. Primero `isdecimal()`, después `int()`.

2 Validar fuera del for

Si el `while` de validación no está **dentro** del `for`, los datos erróneos gastan intentos.

3 El else mal sangrado

El `else` va alineado con el `for`, no con el `if`: si no, sale «Sin intentos» en cada fallo.

4 Al mejor de 3 = 3 rondas

Hay que jugar hasta que alguien llegue a 2 victorias: los empates no cuentan.

Lo que viene: funciones

SE REPITE 4 VECES EN LA P2

```
while True:
    texto = input("…").strip()
    if …:           # ¿es válido?
        break
    print("…")      # aviso
```

Funciones

Un bloque con **nombre** que recibe datos y devuelve un resultado.

```
pedir_entero(mensaje, minimo, maximo)
ganador_ronda(jugada, ordenador)
```

Guarda tu P2. Al empezar las funciones partiremos de tu respuesta a la Parte D para refactorizarla.